

Sistemas Distribuidos

GRADO EN INGENIERÍA INFORMÁTICA



MEMORIA

Ejercicio evaluable 1: Sistemas Distribuidos

Curso 2025/2026

DATOS DE LOS AUTORES

Nombre:	David Sierra
NIA:	100471913
Correo electrónico:	100471913@alumnos.uc3m.es

Nombre:	Juan Luis del Valle Caetano
NIA:	100454201
Correo electrónico:	100454201@alumbos.uc3m.es

1. Diseño e implementación

1.1. Parte A. Versión no distribuida

Se estructura en tres componentes: el archivo de cabecera *claves.h*, que define la API pública y la estructura Paquete; la biblioteca dinámica *libclaves.so*, compilada a partir de *claves.c*; y el ejecutable *app-cliente*, enlazado con dicha biblioteca. Las tuplas <key, value1, value2, value3> se almacenan en una lista enlazada dinámica. Cada nodo contiene la clave (hasta 255 caracteres), value1 (hasta 255 caracteres), el vector de floats con su dimensión N (entre 1 y 32 elementos), y la estructura Paquete {x, y, z}. Todas las operaciones sobre la lista se protegen mediante un mutex POSIX (`pthread_mutex_t`), inicializado estáticamente con `PTHREAD_MUTEX_INITIALIZER`. Esto garantiza que cada operación se ejecuta de forma atómica, evitando condiciones de carrera cuando múltiples hilos acceden simultáneamente a la estructura de datos.

1.2. Parte B. Versión distribuida con colas de mensajes POSIX

La versión distribuida mantiene exactamente la misma API que la parte A. El cliente utiliza *libproxyclaves.so* (compilada desde *proxy-mq.c*), que intercepta cada llamada a la API y la transmite al servidor mediante colas de mensajes POSIX. El servidor (*servidor_mq*) recibe las peticiones, invoca las funciones de *libclaves.so* y devuelve la respuesta al cliente. Se definen dos estructuras en *protocolo.h*: **Peticion** (cliente → servidor) y **Respuesta** (servidor → cliente). La estructura **Peticion** contiene el tipo de operación, el PID del cliente para identificar su cola de respuesta privada, y los parámetros de la operación. La estructura **Respuesta** contiene el código de retorno y, en el caso de `get_value`, los datos recuperados.

Se utilizan dos tipos de colas:

- `/cola_servidor_sd`: cola única del servidor, creada al arrancar y destruida al terminar.
- `/cola_cliente_sd_<PID>`: cola privada por proceso cliente, creada en la primera llamada a la API y eliminada automáticamente al terminar el proceso mediante un destructor (`__attribute__((destructor))`).

El servidor crea un hilo POSIX por cada petición recibida, en modo *detached*, por lo que no es necesario hacer `join`. Se eligió este mecanismo frente a un *pool de hilos* por las siguientes razones:

- **Simplicidad de implementación**: no requiere gestionar una cola de trabajo ni sincronizar workers, lo que reduce la complejidad del código y los posibles puntos de fallo.
- **Adecuación a la carga**: dado que la carga esperada es moderada y las operaciones sobre la lista enlazada son muy rápidas, el coste de crear un hilo por petición es despreciable.
- **Sin límite artificial**: un pool de hilos fijo limitaría el número de peticiones concurrentes. Con hilos bajo demanda, el servidor puede atender tantas peticiones simultáneas como permita el sistema.

El acceso a la estructura de datos compartida sigue protegido por el mutex de *claves.c*, garantizando la atomicidad también en el entorno distribuido con múltiples hilos concurrentes.

Códigos de error

- **0**: operación completada con éxito.

- **-1:** error en la lógica del servicio (clave no encontrada, duplicada, parámetro fuera de rango).
- **-2:** error en las comunicaciones (cola inexistente, timeout de 5 segundos, error de envío/recepción). El proxy valida los parámetros localmente antes de contactar al servidor, devolviendo -1 directamente si value1 supera los 255 caracteres o N_value2 está fuera de rango.

2. Compilación y ejecución

2.1. Compilación

El Makefile genera todos los artefactos necesarios con un único comando:

```
None  
make all
```

Los artefactos generados son los siguientes:

Artefacto	Descripción
libclaves.so	Biblioteca dinámica — versión no distribuida
libproxyclaves.so	Biblioteca dinámica — proxy del cliente (parte B)
app-cliente	Ejecutable cliente enlazado con libclaves.so
app-cliente-mq	Ejecutable cliente enlazado con libproxyclaves.so
servidor_mq	Ejecutable del servidor de colas de mensajes

Para limpiar todos los artefactos generados:

```
None  
make clean
```

2.2. Ejecución

Parte A

```
None  
make all  
./app-cliente
```

Parte B

Requiere dos terminales en el mismo directorio:

```
None
# Terminal 1: arrancar el servidor
./servidor_mq

# Terminal 2: ejecutar el cliente
./app-cliente-mq
```

Si las colas quedan en estado inconsistente tras un fallo:

```
None
rm -f /dev/mqueue/cola_servidor_sd
/dev/mqueue/cola_cliente_sd_*
```

3. Protocolo de comunicación Cliente-servidor

CLIENTE (proxy-mq.c)	SERVIDOR (servidor-mq.c)
-- valida parámetros ----->	
(localmente, sin contactar	
al servidor si hay error)	
-- crea cola /cola_cliente_PID	
-- mq_send(Peticion) ----->	
{operacion, PID, key,	
value1, N, V, paquete}	
	crea hilo ----->
	invoca funcion
	libclaves.so
<-- mq_send(Respuesta) -----	
{resultado, value1,	
N, V, paquete}	
-- mq_timedreceive()	
(timeout 5 segundos)	
-- devuelve resultado	
al código del cliente	

Notas:

1. Si `mq_timedreceive` expira sin respuesta, el proxy devuelve -2.
2. La cola del cliente se elimina automáticamente al terminar el proceso.
3. Cada petición al servidor genera un hilo independiente en modo *detached*.

4. Plan de pruebas

El plan se implementa en `app-cliente.c` y es idéntico para ambas versiones. Consta de 27 comprobaciones agrupadas en 15 pruebas:

PRUEBA	RESULTADO OBTENIDO
<code>destroy()</code> inicial	0
<code>set_value</code> con datos válidos (x2)	0
<code>set_value</code> con clave duplicada	-1
<code>set_value</code> con N=0 y N=33	-1
<code>exist</code> con clave existente e inexistente	1/0
<code>get_value</code> + verificación de los 5 campos	0
<code>get_value</code> con clave inexistente	-1
<code>modify_value</code> + verificación de modificación	0
<code>modify_value</code> con clave inexistente	-1
<code>delete_key</code> + <code>exist</code> tras borrar	0/0
<code>delete_key</code> con clave inexistente	-1
Clave de 256 chars, value1 de 256 chars, clave de 255 chars	-1/-1/0
<code>set_value</code> y <code>get_value</code> con N=32	0
10 inserciones + verificación de existencia	0
<code>destroy()</code> + verificación de limpieza	0

Resultado obtenido: 27/27 pruebas exitosas en ambas versiones, tanto en local como en Guernika.